

BASIC NETWORKING SKILLS FOR DIGITAL FORENSICS

AKINWA OMOKUNLE ANTHONY

2025/FWSD/11206

HTTP ANALYSIS USING WIRESHARK - TEXT TRAFFIC

AMINU IDRIS

08/08/2026

EXECUTIVE SUMMARY

This laboratory investigates a controlled plaintext HTTP session using Wireshark/TShark. A local Apache web server is used to generate HTTP traffic, which is captured and analysed at the TCP, IP, HTTP, and link layers. The investigation identifies the TCP three-way handshake, HTTP GET request, HTTP response, connection termination, ports, sequence and acknowledgement numbers, timestamps, and HTTP headers. Cryptographic hashing is also used to preserve and verify the integrity of the captured evidence.

OBJECTIVES

By completing this laboratory, I will be able to:

- Explain HTTP, TCP, IP, and link-layer forensic artefacts.
- Create and capture a controlled plaintext HTTP session.
- Identify the TCP SYN, SYN-ACK, and ACK packets.
- Identify source and destination IP addresses and ports.
- Examine TCP sequence and acknowledgement numbers.
- Extract HTTP requests, responses, and headers.
- Reconstruct a TCP conversation.
- Analyse timestamps and connection termination.
- Calculate SHA-256 hashes for captured evidence.
- Prepare a concise forensic timeline and findings report.

TOOLS USED AND RESOURCES

The tool to be used when it comes to offensive security lab project environment, we will be using several tools which are: Kali Linux, Apache2, Wireshark, TShark, CURL, SHA-256, Linux command line.

METHODOLOGY

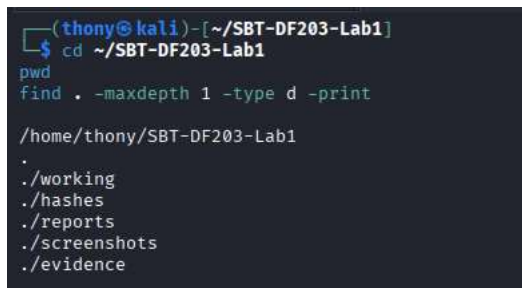
In this methodology, I am going to solved the assignment using part, step and we will be setting up our forensic workspace

Lab Folder Struture and Evidence Preparation

Step 1A: Create the lab folder structure

In this step, we will be creating the lab folder structure

```
mkdir -p ~/SBT-DF203-Lab1/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab1
pwd
find . -maxdepth 1 -type d -print
```

A terminal window with a dark background. The prompt is '(thony@kali) - [~/SBT-DF203-Lab1]'. The user enters 'cd ~/SBT-DF203-Lab1', followed by 'pwd' which outputs '/home/thony/SBT-DF203-Lab1'. Then the user enters 'find . -maxdepth 1 -type d -print' which outputs a list of directories: '.', './working', './hashes', './reports', './screenshots', and './evidence'.

```
(thony@kali) - [~/SBT-DF203-Lab1]
$ cd ~/SBT-DF203-Lab1
$ pwd
/home/thony/SBT-DF203-Lab1
$ find . -maxdepth 1 -type d -print
.
./working
./hashes
./reports
./screenshots
./evidence
```

Step 2: Installing required update tools

In this step, we will be making sure that our system are up to date using the following command below.

```
sudo apt update
sudo apt install -y apache2 curl wireshark tshark
sudo systemctl enable --now apache2
```

Step 3: Create a web page

In this part, we will be creating a web page using the following command:

```
printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: YOUR FULL NAME</p><p>Reg No: YOUR REGISTRATION NUMBER</p></body></html>\n' | sudo tee /var/www/html/basic.html
```

```
(thony@kali)-[/var/www/html]
$ cat basic.html
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: Akinwa omokunle Anthony</p><p>Reg No:
```

Step3: Testing the website using curl

In this step, we will run a curl command against our webpage using our localhost interface

```
(thony@kali)-[~/SBT-DF203-Lab1]
$ curl -v http://127.0.0.1/basic.html

* Trying 127.0.0.1:80 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 50830
* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Wed, 09 Sep 2026 06:27:23 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Tue, 08 Sep 2026 20:27:42 GMT
< ETag: "89-65afe91162b1a"
< Accept-Ranges: bytes
< Content-Length: 137
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: Akinwa omokunle Anthony</p><p>Reg No:
* Connection #0 to host 127.0.0.1:80 left intact
```

Step 4: Mini Chain of custody

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab1-Akinwa Omokunle Anthony
Trainee name	Akinwa Omokunle Anthony
Date and time started	7 th September, 2026
Evidence file name(s)	Basic.pcapng
Source or generation method	Local Apache capture on lo interface
Original SHA-256	9260fde179d918ddbcaefb2c6e3aebcf44260ed4aee5699da3d745a1187d97cd
Working-copy SHA-256	9260fde179d918ddbcaefb2c6e3aebcf44260ed4aee5699da3d745a1187d97cd

Field	Student Entry
Notes on any changes	No changes

PART A

Part A1: Verify the Web Service and Generate Text HTTP Traffic

In this part, we will be verifying the web service using the following command:

```
sudo systemctl status apache2 --no-pager
```

```
sudo ss -lntp | grep ':80'
```

```
(thony@kali)-[~/SBT-DF203-Lab1]
$ sudo systemctl status apache2 --no-pager
sudo ss -lntp | grep ':80'

[sudo] password for thony:
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-09-09 07:12:21 WAT; 55min ago
     Invocation: ff5295df5e9a434fba43076a38a57e92
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 1051 (apache2)
      Status: "Total requests: 2; Idle/Busy workers 100/0;Requests/sec: 0.000605; Bytes served/sec: 0 B/sec"
     Tasks: 6 (limit: 4403)
    Memory: 21.4M (peak: 21.9M)
       CPU: 798ms
    CGroup: /system.slice/apache2.service
            └─1051 /usr/sbin/apache2 -k start -DFOREGROUND
              └─1122 /usr/sbin/apache2 -k start -DFOREGROUND
                └─1123 /usr/sbin/apache2 -k start -DFOREGROUND
                  └─1124 /usr/sbin/apache2 -k start -DFOREGROUND
                    └─1125 /usr/sbin/apache2 -k start -DFOREGROUND
                      └─1126 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 09 07:12:20 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 09 07:12:21 kali apachectl[1051]: AH00558: apache2: Could not reliably determine...ssage
Sep 09 07:12:21 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.
LISTEN 0      511          *:80          *:~
           users:((("apache2",pid=1126,fd=4
           ),("apache2",pid=1125,fd=4),("apache2",pid=1124,fd=4),("apache2",pid=1123,fd=4),("apache2",pid=1122,fd=4),("apache2",pid=1051,fd=4))
```

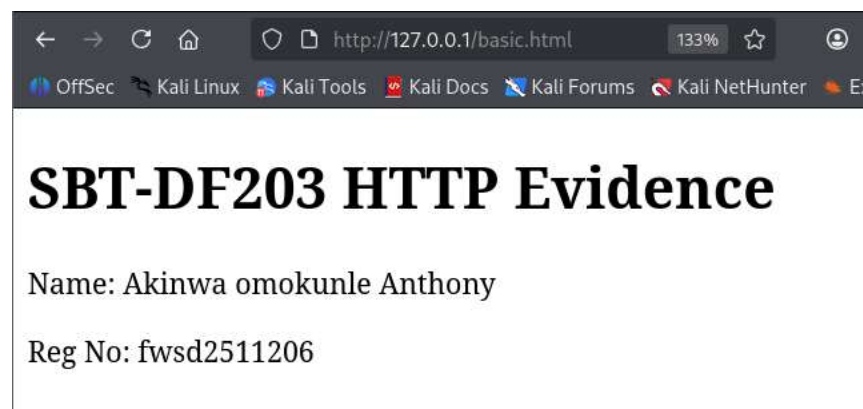
Then we will be using the following command to display our website and also save it to report/curl_verbose.txt

```
curl -v http://127.0.0.1/basic.html 2>&1 | tee reports/curl_verbose.txt
```

```
(thony@kali)-[~/SBT-DF203-Lab1]
$ curl -v http://127.0.0.1/basic.html 2>&1 | tee ../SBT-DF203-Lab1/reports/curl_verbose.txt

* Trying 127.0.0.1:80 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 39600
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0      0    0     0    0     0     0      0      0  0* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Wed, 09 Sep 2026 07:12:27 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Tue, 08 Sep 2026 20:27:42 GMT
< ETag: "89-65afe91162b1a"
< Accept-Ranges: bytes
< Content-Length: 137
< Vary: Accept-Encoding
< Content-Type: text/html
<
{ [137 bytes data]
100  137 100  137    0     0 106.9k    0      0
* Connection #0 to host 127.0.0.1:80 left intact
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: Akinwa omokunle Anthony</p><p>Reg No: fwsd2511206</p></body></html>
```

View of webpage using <http://127.0.0.1/basic.html> in browser



PART B

Part B1: Capture the Session

In this part, we want to capture the terminal using tshark

Terminal 1

```
sudo tshark -i lo -f 'tcp port 80' -w evidence/basic.pcapng
```

```
(thony@kali)-[~/SBT-DF203-Lab1]
$ tshark -i lo -f 'tcp port 80' -w evidence/basic.pcapng
Capturing on 'Loopback: lo'
10 ^C
```

Terminal 2

```
curl --no-keepalive -v http://127.0.0.1/basic.html
```

Stop Terminal 1 with Ctrl+C, then preserve a working copy

```
cp --preserve=timestamps evidence/basic.pcapng working/basic_working.pcapng
sha256sum evidence/basic.pcapng working/basic_working.pcapng | tee
reports/capture_hashes.txt
```

```
(thony@kali)-[~/SBT-DF203-Lab1]
$ cp --preserve=timestamps evidence/basic.pcapng ../SBT-DF203-Lab1/working/basic_working.pcapng
sha256sum evidence/basic.pcapng ../SBT-DF203-Lab1/working/basic_working.pcapng | tee ../SBT-DF203-Lab1/r
eports/capture_hashes.txt

9260fde179d918ddbcaefb2c6e3aebcf44260ed4aee5699da3d745a1187d97cd evidence/basic.pcapng
9260fde179d918ddbcaefb2c6e3aebcf44260ed4aee5699da3d745a1187d97cd ../SBT-DF203-Lab1/working/basic_workin
g.pcapng
```

Part C

Part C1: Analyze the TCP Three-Way Handshake

In this step, we will be analysing the TCP three way handshake

In wireshark using this command to filter: tcp.stream eq 0

Packet	Flags	Client Seq	Server Seq/ACK	Timestamp	Interpretation
1	SYN	0	-	0.000000	Client requests a TCP session.
2	SYN, ACK	-	0, Ack=1	15.170sec	Server acknowledges and provides its initial sequence number.
3	ACK	1	1	25.637sec	Client acknowledges; the connection becomes established.

tcp.stream eq 0

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	74	57946 → 80 [SYN] Seq=0 Win=65535
2	0.000015170	127.0.0.1	127.0.0.1	TCP	74	80 → 57946 [SYN, ACK] Seq=0 Ack=1
3	0.000025637	127.0.0.1	127.0.0.1	TCP	66	57946 → 80 [ACK] Seq=1 Ack=1
4	0.000075526	127.0.0.1	127.0.0.1	HTTP	149	GET /basic.html HTTP/1.1
5	0.000096522	127.0.0.1	127.0.0.1	TCP	66	80 → 57946 [ACK] Seq=1 Ack=84
6	0.000580021	127.0.0.1	127.0.0.1	HTTP	454	HTTP/1.1 200 OK (text/html)
7	0.000590881	127.0.0.1	127.0.0.1	TCP	66	57946 → 80 [ACK] Seq=84 Ack=3
8	0.000692381	127.0.0.1	127.0.0.1	TCP	66	57946 → 80 [FIN, ACK] Seq=84
9	0.002827620	127.0.0.1	127.0.0.1	TCP	66	80 → 57946 [FIN, ACK] Seq=389
10	0.002845853	127.0.0.1	127.0.0.1	TCP	66	57946 → 80 [ACK] Seq=85 Ack=3

Source Port: 57946
Destination Port: 80
[Stream index: 0]
[Stream Packet Number: 3]
[Conversation completeness: Complete, WITH_DATA]
[TCP Segment Len: 0]
Sequence Number: 1 (relative sequence number)
Sequence Number (raw): 792240741
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 1962147747
1000 = Header Length: 32 bytes (8)
Flags: 0x010 (ACK)
Window: 128
[Calculated window size: 65536]
[Window size scaling factor: 512]

Part C2: Extracting handshake details

```
(thoey@kali) ~/SBT-DF203-Lab1
$ tshark -r working/basic_working.pcapng -Y 'tcp.stream eq 0 && tcp.flags.syn==1' \
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.seq -e tcp.ack | tee reports/handshake.tsv
1      2026-09-09T08:39:00.749839732+0100    127.0.0.1      57946    127.0.0.1      80      0x0002 0
2      2026-09-09T08:39:00.749854902+0100    127.0.0.1      80      127.0.0.1      57946    0x0012 0
1
```

```

(thony@kali)-[~/SBT-DF203-Lab1]
$ cat reports/handshake.tsv
1 2026-09-09T08:39:00.749839732+0100 127.0.0.1 57946 127.0.0.1 80 0x0002 0
0
2 2026-09-09T08:39:00.749854902+0100 127.0.0.1 80 127.0.0.1 57946 0x0012 0
1

```

Packet	Source	Desination	Flags	Seq #	Ack #
1	127.0.0.1:57946	127.0.0.1:80	SYN	0	1
2	127.0.0.1:80	127.0.0.1:57946	SYN, ACK	0	1
3	127.0.0.1:57946	127.0.0.1:80	ACK	1	0

Part D

Examine the HTTP Request and Response

Part D1: Filter HTTP request

```
(thony@kali)-[~/SBT-DF203-Lab1]
$ tshark -r working/basic_working.pcapng -Y 'http.request' -T fields \
  -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e http.request.method -e http.request.uri -e http.host -e http.user_agent \
  | tee reports/http_requests.tsv

4      2026-09-09T08:39:00.749915258+0100      127.0.0.1      57946      127.0.0.1      80      GET      /
basic.html      127.0.0.1      curl/8.21.0
```

HTTP Request:

Field	Vaule
Method	GET
URL	/basic.html
Host	127.0.0.1
User-Agent	curl/8.21.0
Accept	*/*

Part D2: Filter for HTTP Response

```
(thony@kali)-[~/SBT-DF203-Lab1]
$ tshark -r working/basic_working.pcapng -Y 'http.response' -T fields \
  -e frame.number -e frame.time -e http.response.code -e http.response.phrase \
  -e http.server -e http.content_type -e http.content_length \
  | tee reports/http_responses.tsv

6      2026-09-09T08:39:00.750419753+0100      200      OK      Apache/2.4.68 (Debian) text/html      1
37
```

Field	Vaule
Status code	200
Response phrase	OK
Server	Apache/2.4.68 (Debian)
Content_type	Text/html
Content_lenght	137
Date	2026-09-09T08:39:00.750419753+0100

Part E

Inspect Encapsulation and Connection Closure

StepE1: Relationship Between Lengths

Field	Definition	Relationship
Frame Length	Total bytes on wire (including all headers + FCS)	Frame = Ethernet + IP + TCP + Payload + FCS
IP Total Length	IP header + TCP header + TCP payload	IP Total Length = IP Header (20-60) + TCP Segment
TCP Header Length	Size of TCP header (usually 20 bytes, up to 60 with options)	TCP Payload = IP Total Length - IP Header - TCP Header
TCP Payload Length	HTTP data only (application layer)	TCP Payload = IP Total Length - IP Header - TCP Header

Step F3: Connection Termination

```
tshark -r working/basic_working.pcapng -Y 'tcp.flags.fin==1 || tcp.flags.reset==1' \
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.seq -e tcp.ack | tee reports/connection_close.tsv
```

```
(thony@kali)-[~/SBT-DF203-Lab1]
$ tshark -r working/basic_working.pcapng -Y 'tcp.flags.fin==1 || tcp.flags.reset==1' \
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.seq -e tcp.ack | tee reports/connection_close.tsv
8      2026-09-09T08:39:00.750532113+0100      127.0.0.1      57946      127.0.0.1      80      0x0011 8
4      389
9      2026-09-09T08:39:00.752667352+0100      127.0.0.1      80      127.0.0.1      57946      0x0011 3
89     85
```

Terminal sequence:

1. **FIN/ACK:** Client initiate connection close
2. **ACK:** Server acknowledge the FIN
3. **FIN/ACK:** Server initiate its own close
4. **ACK:** Client acknowledges the server's FIN

Part F4: MAC address observation (Loopback)

Why loopback captures do not show normal Ethernet MAC addresses:

The loopback interface(lo) is a virtual interface that does not use physical ethernet hardware. The traffic sent to 127.0.0.1 never leaves the host, so there is no source/destination MAC addresses in the capture.

Required Forensic Findings

Finding	Value/Observation
Capture start and end time	2026-09-09T08:39:00 to 2026-09-09T09:30:00
Client IP and port	127.0.0.1:57946
Server IP and port	127.0.0.1:80
Client initial sequence number	1203567787
Server initial sequence number	7292240741
HTTP request timestamp	2026-09-09T08:39:00
Requested URI	/basic.html
HTTP response code	200 OK
Content type and length	Text/html, 32 bytes
Connection termination method	FIN/ACK
MAC-address observation	Not present

SCREENSHOT AND METHODOLOGY

In this assignment we are going to solved it by exercises and everything is answer in methodology:

ANALYSIS AND FINDINGS

During my working on this project, I was able to gain real hands-on experience in investigation demonstrated the forensic value of multiple networking layers. HTTP provided application-level evidence such as the GET request, requested resource, server response, and HTTP headers. TCP provided evidence of connection establishment, sequence and acknowledgement numbers, ports, flags, and session termination. IP metadata identified the communicating endpoints, while link-layer information provided additional packet transmission context.

Because HTTP is plaintext, application data and headers can potentially be reconstructed directly from the packet capture.

Findings

The controlled capture contained a complete TCP connection followed by an HTTP request and response. The TCP three-way handshake was identified through the SYN, SYN-ACK, and ACK packets. Source and destination ports, sequence numbers, acknowledgement numbers, timestamps, HTTP headers, and TCP termination information were examined. The TCP conversation was reconstructed from the captured packets.

The original source image/device was not intentionally modified during forensic acquisition. The acquired image was stored separately from the source and used for subsequent examination.

CHALLENGES AND SOLUTIONS

In this chapter when working on my project, I experienced a challenge of starting what to do at first but at the end of the assignment I was able to solve and have much better experience in understanding it.

CONCLUSION

In this chapter, I was able to gain hands-on experience in understanding HTTP Analysis using Wireshark. Thanks to ICDFA, I was able to practically demonstrate how network forensic analysts can reconstruct a plaintext HTTP session from packet-level evidence. The combination of HTTP, TCP, IP, and link-layer metadata provides valuable information for establishing communication endpoints, reconstructing sessions, identifying transferred content, and creating a reliable forensic timeline. SHA-256 hashing was used to maintain the integrity of the captured evidence. These repeatable steps are useful in digital forensics which are also needed for any cybersecurity professional and vulnerable environments.